Part 2 Onboarding Guide

Checkout and Orders | POS Project (video game store)

Prepared Oct 19, 2026 (Day 4 of the plan). Based on the Part Briefs (Oct 16), the current schema design document (updated Oct 16, evening) and the project plan.

1. At a glance

Item	Detail
Your part	Part 2: Checkout and orders. This is the **critical path**: returns, loyalty points and the sales report all depend on completed sales existing.
Branch	`part-2-checkout`. Pull requests go to the lead, who merges to main. Vercel deploys from main only.
Folders you own	`db/checkout/`, `api/checkout/`, `api/orders/`, `api/reports/sales.js`, `src/pages/checkout/`, and your own sales report file in `src/pages/reports/`.
You build	3 triggers, the stored procedure `sp_Checkout`, SQL tests, 3 API areas (checkout, orders, sales report), 5 screens, and a Jest test per endpoint.
Deadlines	Triggers and `sp_Checkout`: end of Day 3 (Oct 18), so these are already due. Walking skeleton (log in, browse, add to cart, pay, see the order, live on Vercel): end of Day 4 (Oct 19). Feature freeze: Oct 23. Submission: Oct 25.
Never cut	Checkout with its three triggers, login and roles, the sales report. If you fall behind, tell the lead early: Part 2 is the first part the lead takes over.

2. Documents to read first

Document	How to use it
schema-current-design.md	**Source of truth** for tables, columns, triggers, decisions and the rebuild procedure. Newer than the conceptual schema PDF.
POS Project: Part Briefs (PDF)	Your Part 2 page plus the shared rules, the lead's deliverables, the timeline and the cut order.
Conceptual schema PDF	**Outdated. Ignore it for design.** It still has Register, Cashier, nullable Customer_id on transactions and cash payments, which were all removed.
Technology resources PDF	Tutorials for SQL Server, Node.js and React. Useful for learning, not for decisions.
triggers.sql, test_triggers.sql, drop.sql, schema file	SQL files you already have. `triggers.sql` shows the exact style your three triggers must follow. `test_triggers.sql` is the model for your `db/checkout/test.sql`.

3. Install the tools

These steps assume Windows (SQL Server Express runs on Windows). Run commands in PowerShell or the VS Code terminal (Ctrl+`). You already have VS Code and a SQL Server Express connection from VS Code, so those are skipped.

Step 1: Git

1. Download Git for Windows from https://git-scm.com/download/win and run the installer with the default options.
2. Close and reopen VS Code, then run `git --version`.
3. Set your identity (use your GitHub email):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 2: Node.js (LTS)

1. Download the **LTS** version from https://nodejs.org (not Current) and install with the defaults.
2. Reopen VS Code and check `node -v` and `npm -v`. Both must print a version.
3. If PowerShell says scripts are disabled when you run npm, run this once:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Step 3: Get the project and install its packages

> **Important.** Do **not** run `npm create vite` or start a new project. The lead owns the skeleton (Vite, React Router, lib/, src/shared/). You work inside the shared repo.

1. The team repository already exists and you have access. Clone it (use the repository URL from GitHub, under the green Code button) and move into the folder.
2. Create your branch.
3. Install everything listed in the lead's `package.json`.
4. Check the stack is present.

```
cd C:\Projects
git clone <repo-url>
cd <repo-folder>
git checkout -b part-2-checkout
npm install
npm ls mssql bcrypt jsonwebtoken jest react-router-dom vite
```

- If a package is missing, **tell the lead** instead of committing a change to `package.json`, so the five parts do not conflict. To keep testing locally in the meantime: `npm install mssql bcrypt jsonwebtoken react-router-dom` and `npm install --save-dev jest vite`.
- If `bcrypt` fails to install (it is a native module), the lead can switch to `bcryptjs`. Otherwise install "Desktop development with C++" from the Visual Studio Build Tools and retry.

Step 4: Let Node connect to your local SQL Express

VS Code's SQL extension can connect in ways Node's `mssql` package cannot, so this step is often the real blocker.

1. Open **SQL Server Configuration Manager**. Under SQL Server Network Configuration, Protocols for SQLEXPRESS, right-click **TCP/IP** and choose Enable.
2. Under SQL Server Services, set **SQL Server Browser** to Automatic and Running (needed to find the named instance).
3. Restart **SQL Server (SQLEXPRESS)** in the same list.
4. Node needs a SQL login, not Windows authentication. In VS Code, on `master`, check the mode. A result of 0 means mixed mode is already on. If it returns 1, enable "SQL Server and Windows Authentication mode" in the server Properties, Security page, then restart the service.

```
SELECT SERVERPROPERTY('IsIntegratedSecurityOnly');
```

5. Create a local development login. This is for your machine only; choose your own password.

```
CREATE LOGIN pos_dev WITH PASSWORD = 'ChooseAStrongPassword!1';
GO
USE pos_store;
GO
CREATE USER pos_dev FOR LOGIN pos_dev;
ALTER ROLE db_owner ADD MEMBER pos_dev;
GO
```

6. Put your settings in `.env.local` in the repo root. Ask the lead for the exact variable names that `lib/db.js` reads, and make sure `.env*` is in `.gitignore`. **Never commit this file.**

Setting	Local value
Server	localhost
Instance	SQLEXPRESS
Database	pos_store
User / password	The pos_dev login you created
Encryption	encrypt false and trust server certificate true (local only; the Azure VM uses encrypt: true)

Step 5: Vercel CLI (optional)

It runs the `api/` functions locally the way they run in production. The functions are plain Node handlers, not Express.

1. Install and log in: `npm install -g vercel`, then `vercel login`.
2. The lead must first add you to the Vercel project. Then run `vercel link` inside the repo.
3. Do **not** pull production environment variables unless the lead says so, because that downloads real credentials. Use your own `.env.local`.
4. Run `vercel dev` for app and API together. If the lead's setup uses `npm run dev` for the front end only, use that for screen work and rely on Jest for the API.

Step 6: Optional VS Code extensions

- **ESLint** and **Prettier**, if the repo uses them.
- **Thunder Client** or **REST Client**, to call `api/checkout` without a screen.

Step 7: Verify the setup

[ ]	`git --version`, `node -v` and `npm -v` each print a version.
[ ]	`git branch` shows `* part-2-checkout`.
[ ]	`npm ls` shows no missing packages.
[ ]	Nested triggers are on: `SELECT value_in_use FROM sys.configurations WHERE name = 'nested triggers';` returns **1**. The triggers chain, so this is required.
[ ]	`SELECT COUNT(*) FROM sys.tables;` returns 27, and `SELECT name FROM sys.triggers;` lists the 5 built triggers.
[ ]	A small Node script (or the lead's `query()` from `lib/db.js`, if it exists) connects as `pos_dev` and runs `SELECT 1`.
[ ]	`npx jest` runs without crashing ("no tests found" is fine).
[ ]	`npm run dev` starts Vite and the page opens in the browser.

> **Troubleshooting.** If the connection test fails, the usual causes are TCP/IP disabled, SQL Server Browser stopped, mixed mode off, or the service not restarted. Send the exact error message when asking for help.

4. What you need from the lead

[ ]	Vercel project access, if you want to use the CLI (you already have access to the repository).
[ ]	`lib/db.js` (`query(sql, params)` and `withTransaction(fn)`), `lib/auth.js` (`requireAuth(['manager'])(handler)`) and `lib/http.js` (`sendJson`, `sendError`). Names are proposed and become final at the end of Day 2.
[ ]	`src/shared/` components and the API client (`api.get`, `api.post`), plus login and register, so you can test as a customer.
[ ]	The **seed script**: products with stock, customers with a default address, a TAX_RATE for every category in every test state, exactly one reward program with tiers, and completed sales.
[ ]	The exact variable names for `.env.local`, and a least-privilege database login for the Azure VM (not sa) when you need it.
[ ]	The schema file, `triggers.sql`, `test_triggers.sql` and `drop.sql`, if you do not already have all four.
[ ]	Agreement with Part 1 on the **VW_CART columns**. `sp_Checkout` reads the cart through that view.

5. Rules you must follow

Database rules (shared by all parts)

1. Triggers are AFTER, set-based and named `TR_<Table>_<Purpose>`. They start with `SET NOCOUNT ON; SET XACT_ABORT ON;` and fail with `THROW`. Every SQL file begins with `USE pos_store;` then `GO`.
2. Never edit `CUSTOMER.Loyalty_points` or `Tier_id` directly. Points come from a LOYALTY_TRANSACTION row, and the built trigger updates the rest.
3. Never recompute stock, points, tier or low-stock alerts in the app. The built triggers keep them in sync.

4. Every query is parameterized. Prices, tax and totals come from the database, never from the request.
5. A write that touches several tables runs in one database transaction, using the lead's helper.
6. Each trigger's tests go in `db/checkout/test.sql`. Every failing case runs in its own transaction and rolls back.

Decisions already closed (do not reopen)

Topic	Decision
Discounts	Per line, take the **single largest saving** among the product discount and the category discount. Both must have Is_active = 1 and today within Start_date and End_date. No stacking. DISCOUNT.Code is unused (no promo code box). The winner goes into TRANSACTION_ITEM.Discount_id.
Missing tax rate	If any item has no TAX_RATE row for its category, the ship-to state and the sale date, checkout fails with a clear error. **Never fall back to 0%.**
Totals	Subtotal = SUM(Line_total) (after discount, before tax). Total = Subtotal + Tax_amount. No shipping fee and no shipping columns.
Payments	Card only: method is 'credit' or 'debit'. Store only the method and Card_last4. **Never store or log a full card number.**
Customers	No guest checkout. An account is required, and the customer id always comes from the login token, never from the request body.
Request contents	The request carries cart intent, a ship-to choice and payment method only. The database recomputes prices, tax and totals.
Loyalty	Earn only. Points are earned on the Subtotal (not tax) by the existing trigger when the sale completes.

Why Subtotal matters downstream. Part 3's refund formula divides by the sale's Subtotal and assumes Subtotal equals the sum of the line totals. If your Subtotal is wrong, returns will refund the wrong amounts.

6. The data you work with

Figure 1, on the last page of this guide, draws these tables and how they connect. Read the table below for the details of each column, then use the figure to see the relationships.

Table	What Part 2 needs to know
SALES_TRANSACTION	Transaction_id, Customer_id (NOT NULL), Employee_id (nullable), Transaction_date, Subtotal, Tax_amount, Total, Status ('open', 'completed', 'voided'), Ship_name / street / apt / city / state / zip (copied at checkout), Tracking_number (unique when not null), Estimated_arrival. A completed sale requires a ship-to address, tracking number and estimated arrival (CHECK constraint).
TRANSACTION_ITEM	PK (Transaction_id, Product_id). Quantity, Unit_price (price snapshot), Discount_id, Discount_amount, Line_total (persisted computed column).
PAYMENT	Payment_id, Transaction_id, Method ('credit', 'debit'), Amount, Card_last4, Paid_at.
CART_ITEM / VW_CART	CART_ITEM has PK (Customer_id, Product_id), Quantity, Added_at. One cart per customer, no header table. Read it through the view VW_CART (joins PRODUCT, uses Sell_price). Cart rows are deleted at checkout after being copied.

Table	What Part 2 needs to know
PRODUCT	Sell_price, Stock_quantity, Category_id, Discount_id, Is_active. `CHECK (Stock_quantity >= 0)` is what blocks overselling (error 547).
DISCOUNT / CATEGORY	DISCOUNT has Amount, Amount_type ('percent' or 'fixed'), Start_date, End_date, Is_active. Both PRODUCT and CATEGORY point to a discount.
TAX_RATE	Category_id, State, Rate (0 to 1), Effective_from, Effective_to (NULL = open ended). Look up by item category, ship-to state and sale date.
CUSTOMER_ADDRESS	Address_type, Recipient_name, Street, Apt, City, State, Zip, is_default (at most one default per customer). You copy the default address into the sale's Ship_* columns.

What already happens when a sale completes (do not rebuild any of it)

1. `UPDATE SALES_TRANSACTION SET Status = 'completed'` fires **TR_SALES_TRANSACTION_Complete**: subtracts stock and inserts a LOYALTY_TRANSACTION for FLOOR(Subtotal * Points_per_dollar), using the reward program active on the sale date.
2. The stock change fires **TR_PRODUCT_LowStock**, which opens a LOW_STOCK_ALERT when stock is at or below the threshold.
3. The loyalty row fires **TR_LOYALTY_TRANSACTION_Balance**, which updates the customer's points and tier.

Overselling is rejected by the stock CHECK, and because the triggers use XACT_ABORT, a failing sale also undoes its stock decrement. Error numbers 50001 and 50002 are already used by built triggers, so pick different `THROW` numbers.

7. Build order

Do the SQL first, then the API, then the screens. The three triggers and `sp_Checkout` are what unblock Parts 3 and 4 and the lead's integration.

Phase 0: Sync with the lead (before writing code)

- Confirm the VW_CART columns with Part 1.
- Get the seed script and confirm the helper names in lib/.
- Tell the lead the SQL was due end of Day 3 and give an honest ETA.

Phase 1: The three triggers (db/checkout/triggers.sql)

Trigger	Table and event	Rule
TR_TRANSACTION_ITEM_Guard	TRANSACTION_ITEM, AFTER INSERT / UPDATE / DELETE	Items can be added, changed or removed only while the parent sale is 'open'. On insert or update, Quantity must be at most the product's current Stock_quantity.
TR_PAYMENT_Guard	PAYMENT, AFTER INSERT / UPDATE	Payments only on 'open' sales. Amount must be above 0. The sum of a sale's payments cannot exceed its Total.
TR_SALES_TRANSACTION_PaymentCheck	SALES_TRANSACTION, AFTER UPDATE of Status	On 'open' to 'completed': the sale must have at least one item and its payments must add up **exactly** to Total. Otherwise the whole completion is rejected.

- Handle multi-row statements through `inserted` and `deleted`. Use `triggers.sql` as the style model.
- Stock is not reserved, so two open checkouts can both pass the item guard. The stock CHECK at completion remains the final safeguard.
- PaymentCheck and TR_SALES_TRANSACTION_Complete both fire on the same update. Do not rely on which fires first: if PaymentCheck throws, the whole statement, including the stock change, rolls back.
- Do not edit the existing five triggers.

Phase 2: sp_Checkout (one database transaction)

1. Insert the SALES_TRANSACTION as 'open', copying the ship-to address into the Ship_* columns.
2. Copy the cart rows into TRANSACTION_ITEM with the price snapshot and, per line, the single largest saving among the product and category discounts as the discount snapshot. Then delete the cart rows.
3. Set Subtotal, Tax_amount and Total. Tax is looked up per item by category, ship-to state and sale date. If any item has no rate, THROW a clear error.
4. Insert the PAYMENT rows. They must add up exactly to Total.
5. Update the sale to 'completed' with a unique tracking number and an estimated arrival date. A simple rule is enough, for example a generated code and a date 5 days out.

If any step fails (stock, payment mismatch, missing tax rate), everything rolls back, including the stock change made by trigger 1. Totals must be set **before** the payments are inserted, because the payment guard compares against Total.

Decide and write down before coding

- Is a 'fixed' discount amount applied per unit or per line?
- Is tax rounded per line or on the total? (Use one rule consistently.)
- What happens with an empty cart, or an inactive product still in the cart?
- What the customer id and payment inputs look like in the request, and which errors the API shows the user.

Phase 3: Tests (db/checkout/test.sql)

Each failing case runs in **its own transaction** that is rolled back, because a failing statement leaves the transaction unusable.

Required test	Expected result
Edit items on a completed sale	Rejected by TR_TRANSACTION_ITEM_Guard
Overpay a sale	Rejected by TR_PAYMENT_Guard
Complete an unpaid sale	Rejected by TR_SALES_TRANSACTION_PaymentCheck
Oversell (more than the stock)	Fails with error 547

Recommended extras: a happy path (stock drops, points are earned, tier updates); a missing tax rate; an empty cart; the largest discount wins; and a payment total that is one cent off. When done, send `triggers.sql` and `test.sql` to the lead.

Phase 4: Checkout API and screens (walking skeleton)

- `api/checkout/` (POST, customers only). Customer id from the token. Prices, tax and totals always recomputed in the database. Never log a full card number.
- A Jest test that calls the handler directly.
- Checkout screen (address, card, summary) and confirmation screen, each handling loading, error and empty-cart states.

Phase 5: Orders

- `api/orders/` for list and detail. Customers see only their own orders. Old orders still show inactive products.
- Order history and order detail screens. The detail page must include a link to `/returns/new?order=<id>`, which Part 3 is waiting on.
- Jest tests for both endpoints.

Phase 6: Sales report

- `api/reports/sales.js`, managers only. Completed sales only: revenue, orders and units by day and by category, plus top products.
- The report page in `src/pages/reports/` (one file, added only by you).
- A Jest test, including a check that a non-manager is refused. Confirm with the lead whether revenue means Subtotal or Total.

Phase 7: Finish

- Run your triggers and tests together with the lead's chain: a sale should drop stock, open a low-stock alert and earn points.
- Open a pull request from `part-2-checkout` to the lead.

8. Schedule, pacing and cut order

Date	Plan milestone	Suggested pacing for Part 2
Oct 19 (Day 4)	Walking skeleton due	Phases 0 to 3: seed data, triggers, sp_Checkout, tests
Oct 20	Parts 1 to 4 build	Phase 4: checkout API and screens
Oct 21	Parts 1 to 4 build	Phase 5: orders
Oct 22	Parts 1 to 4 build	Phase 6: sales report
Oct 23	**Feature freeze**	Phase 7: full chain test and pull request
Oct 23 to 24	Test and bug bash	Fix bugs found by the team
Oct 25	Submission	One .bak or .sql file (schema, then triggers)

The pacing column is a suggestion built from the plan dates; the milestones in the middle column come from the project plan. The schema freeze was scheduled for the end of Day 3, so do not change tables without the lead's approval.

If you fall behind, cut in this order

1. Order search and filters.
2. Category and month breakdowns in the sales report.
3. A custom ship-to address (use the default address).

Never cut the three triggers, `sp_Checkout` or the sales report.

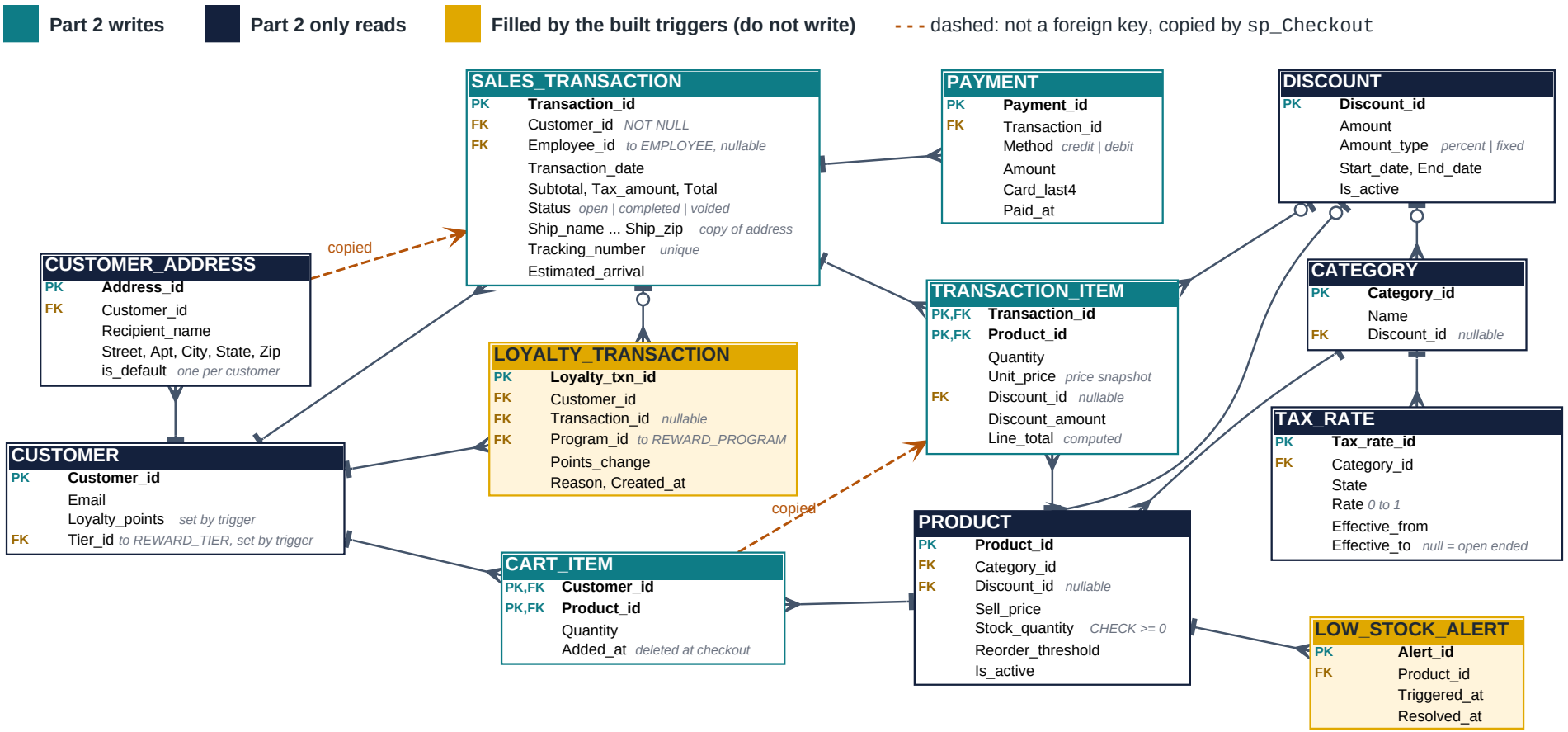
9. Definition of done

- [ ] The three triggers are built, named correctly, set-based, and pass their tests in `db/checkout/test.sql`.
- [ ] `sp_Checkout` completes a sale in one transaction and rolls back fully on any failure.
- [ ] A missing tax rate fails checkout with a clear message.
- [ ] Each endpoint (checkout, orders list and detail, sales report) has a Jest test that calls the handler directly.
- [ ] Each screen handles loading, empty and error states.
- [ ] The order detail page links to `/returns/new?order=<id>`.
- [ ] The sales report is manager only and counts completed sales only.
- [ ] No full card number is stored or logged anywhere, and no secrets are committed.
- [ ] The pull request is open against the lead.

10. Questions to confirm with the lead

- The exact variable names `lib/db.js` reads, and whether `lib/` and the seed script are merged yet.
- The final VW_CART columns (agreed with Part 1).
- Whether sales report revenue means Subtotal or Total.
- How the Azure database login will be shared, and whether the Day 3 trigger deadline has moved (today is Day 4).

Figure 1. The tables behind checkout and how they connect



Reading the lines. A bar means one parent row, a crow's foot means many child rows, and a circle means optional (the foreign key can be NULL). Solid lines are foreign keys. The two dashed lines are not foreign keys: `sp_Checkout` copies the customer's default address into the Ship_* columns of the sale, and copies the cart rows into TRANSACTION_ITEM.

Tax has no foreign key to the sale. `sp_Checkout` looks up TAX_RATE for each item by its category, the ship-to state and the sale date. If no row matches, the checkout must fail.

Order of writes in sp_Checkout. SALES_TRANSACTION as 'open', then TRANSACTION_ITEM (and delete the CART_ITEM rows), then the totals, then PAYMENT, then update the sale to 'completed'. That last update fires the built triggers, which change PRODUCT stock, LOW_STOCK_ALERT, LOYALTY_TRANSACTION and the customer's points and tier. Do not do any of that yourself.

Not drawn. EMPLOYEE, REWARD_PROGRAM and REWARD_TIER appear only as notes ("to EMPLOYEE", "to REWARD_TIER", "to REWARD_PROGRAM"). Part 2 does not write them, but the seed data needs exactly one reward program with tiers, because the loyalty trigger relies on it.